Assignment: EC2 (PHP) → S3 → Lambda → SNS Email Notification

Architecture

```
Student's browser
      |
      v
EC2 instance (PHP app: index.html + s3upload.php)
      |  (PutObject via AWS SDK for PHP)
      v
S3 Bucket
      |  (ObjectCreated event trigger)
      v
Lambda (Python) — reads object metadata via HeadObject
      |  (sns.publish)
      v
SNS Topic  →  Email to all subscribers
```

Files

- `php-ec2-app/index.html` — upload form
- `php-ec2-app/s3upload.php` — handles the upload to S3
- `lambda/lambda_function.py` — reads metadata, publishes to SNS

Step 1 — Create the S3 bucket

1. S3 Console → Create bucket → give it a unique name (e.g. `student-uploads-yourname`).
2. Keep default settings (block public access ON is fine — the app uses IAM, not public URLs).

Step 2 — Create the SNS topic and add subscribers

1. SNS Console → Topics → Create topic → type **Standard** → name it, e.g. `s3-upload-notifications`.
2. Open the topic → Create subscription → Protocol: **Email** → Endpoint: each student/user's email address.
3. Each subscriber must confirm the subscription via the confirmation email AWS sends them — SNS won't deliver until confirmed.
4. Copy the Topic ARN — you'll need it in Step 3.

Step 3 — Create the Lambda function

1. Lambda Console → Create function → Author from scratch → Runtime: Python 3.12.
2. Paste in `lambda_function.py`.
3. Configuration → Environment variables → add:
    - `SNS_TOPIC_ARN` = the ARN copied above

4. Configuration → Permissions → open the execution role → attach an inline policy allowing:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    },
    {
      "Effect": "Allow",
      "Action": ["sns:Publish"],
      "Resource": "YOUR-SNS-TOPIC-ARN"
    }
  ]
}
```

Step 4 — Connect S3 to Lambda (the trigger)

1. Open your S3 bucket → Properties → Event notifications → Create event notification.
2. Event name: `on-upload`.
3. Event type: **All object create events** (`s3:ObjectCreated:*`).
4. Destination: Lambda function → select the function from Step 3.
5. Save. (S3 automatically adds the required resource-based permission to the Lambda function.)

Step 5 — Launch the EC2 instance and deploy the PHP app

1. Launch an EC2 instance (Amazon Linux 2023 or Ubuntu), open port 80 (HTTP) in the security group.
2. Install PHP and a web server, e.g. on Amazon Linux:

```
sudo dnf install -y php php-cli httpd unzip
sudo systemctl enable --now httpd
```

3. Install Composer and the AWS SDK for PHP inside your web root (`/var/www/html`):

```
cd /var/www/html
curl -sS https://getcomposer.org/installer | php
php composer.phar require aws/aws-sdk-php
```

4. Copy `index.html` and `s3upload.php` into `/var/www/html/`.
5. Edit `s3upload.php` and set `$bucket` and `$region` to match Step 1.

Step 6 — Give EC2 permission to upload to S3 (no hardcoded keys!)

1. IAM Console → Roles → Create role → Trusted entity: EC2.
2. Attach a policy allowing `s3:PutObject` on your bucket (same pattern as the Lambda policy above, but `PutObject` instead of `GetObject`).
3. Attach the role to your EC2 instance: EC2 Console → select instance → Actions → Security → Modify IAM role.

Step 7 — Test end to end

1. Visit `http://<EC2-public-IP>/index.html` in a browser.
2. Upload a file.
3. Check your email — you should receive an SNS notification within a few seconds containing the object's size, content type, and metadata.

Things to discuss with students

- Why we use an **IAM Role** on EC2/Lambda instead of hardcoded access keys.
- The difference between S3 **system metadata** (size, content-type) and **custom user metadata** (the `Metadata` array sent at upload time).
- How S3 event notifications decouple the upload from the notification logic (event-driven architecture).
- SNS fan-out: the same topic could also trigger an SQS queue or another Lambda in parallel with the email.